Lab.2

Zadanie 1

Program tworzy nowy proces funkcją fork(). Proces dziecka (fork() zwraca 0) wypisuje „Hello from child”, a proces rodzica (fork() zwraca PID > 0) wypisuje „Hello from parent”.

```
 GNU nano 7.2
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid = fork();

if (pid == 0)
{
   printf("Hello from child\n");
} else {
   printf("Hello from parent\n");
}
return 0;
}
```

```
root@debian:~# gcc zad1L2.c -o zad1L2
root@debian:~# ./zad1
Zapisano 10 liczb!
root@debian:~# ./zad1L2
Hello from parent
root@debian:~# Hello from child
```

Zadanie 2

Proces dziecka liczy sumę liczb od 1 do 10 w pętli i wypisuje wynik. Proces rodzica czeka na zakończenie dziecka funkcją wait(), a dopiero potem kończy działanie.

```
 GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
pid_t pid = fork();

if (pid == 0)
{
int sum = 0;
for (int i = 1; i <= 10; i++)
    sum += 1;
printf("Dziecko: suma = %d\n", sum);
} else {
  wait(NULL);
  printf("Rodzic: dziecko skonczylo\n");
}
return 0;
}
```

```
root@debian:~# gcc zad2L2.c -o zad2L2
root@debian:~# ./zad2L2
Dziecko: suma = 10
Rodzic: dziecko skonczylo
root@debian:~#
```

Zadanie 3

Program tworzy w pętli dwa procesy potomne. Każde dziecko wypisuje swój PID (getpid()) oraz swój numer kolejny, a rodzic czeka na oba procesy (dwa wywołania wait()) i wypisuje komunikat po ich zakończeniu.

```
 GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
for (int i = 1; i <= 2; i++)
{
pid_t pid = fork();
if (pid == 0)
{
printf("Dziecko %d, PID: %d\n", i, getpid());
return 0;
      }
}
wait(NULL);
wait(NULL);
printf("Rodzic: oba dzieci skonczone\n");
return 0;
}
```

```
root@debian:~# gcc -std=c99 zad3L2.c -o zad3L2
root@debian:~# ./zad3L2
Dziecko 2, PID: 584
Dziecko 1, PID: 583
Rodzic: oba dzieci skonczone
root@debian:~# _
```

Zadanie 4

Program po wykonaniu fork() wypisuje PID rodzica i dziecka. Użyto funkcji **getpid()** (zwraca PID bieżącego procesu) oraz **getppid()** (zwraca PID procesu rodzica) dzięki temu dziecko może pokazać zarówno swój PID, jak i PID rodzica.

```
 GNU nano 7.2
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid = fork();

if (pid == 0)
{
printf("Dziecko - PID: %d\n", getpid());
printf("Dziecko - PID rodzica: %d\n", getppid());
} else {
printf("Rodzic - PID: %d\n", getpid());
printf("Rodzic - PID dziecka: %d\n", pid);
}
return 0;
}_
```

```
root@debian:~# gcc zad4L2.c -o zad4L2
root@debian:~# ./zad4L2
Rodzic - PID: 597
Rodzic - PID dziecka: 598
root@debian:~# Dziecko - PID: 598
Dziecko - PID rodzica: 1

root@debian:~#
```

Zadanie 5

Proces dziecka kończy działanie funkcją exit() z wybranym kodem (np. exit(42)). Rodzic odczytuje status zakończenia przez wait(&status), a następnie sprawdza WIFEXITED(status) (czy dziecko zakończyło się normalnie) i WEXITSTATUS(status) (kod wyjścia dziecka).

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
pid_t pid = fork();

if (pid == 0)
{
printf("Dziecko: koncze z kodem 42\n");
exit(42);
} else {
int status;
wait(&status);
if (WIFEXITED(status))
{
printf("Rodzic: dziecko skonczylo\n");
printf("Rodzic: kod wyjscia = %d\n", WEXITSTATUS(status));
            }
        }
return 0;
}
```

```
root@debian:~# gcc zad5L2.c -o zad5L2
root@debian:~# ./zad5L2
Dziecko: koncze z kodem 42
Rodzic: dziecko skonczylo
Rodzic: kod wyjscia = 42
root@debian:~#
```

Zadanie 6

Proces dziecka wypisuje liczby od 1 do 5 z opóźnieniem 1 sekundy (sleep(1)), a rodzic wypisuje liczby od 6 do 10. W terminalu liczby z obu procesów przeplatają się, ponieważ procesy działają równolegle (asynchronicznie), a kolejność zależy od planisty systemu.

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
pid_t pid = fork();

if (pid == 0)
{
for (int i = 1; i <= 5; i++)
{
printf("Dziecko: %d\n", i);
sleep(1);
    }
} else {
for (int i = 6; i <= 10; i++)
{
printf("Rodzic: %d\n", i);
sleep(1);
    }
}
return 0;
}
```

```
root@debian:~# gcc -std=c99 zad6L2.c -o zad6L2
root@debian:~# ./zad6L2
Rodzic: 6
Dziecko: 1
Rodzic: 7
Dziecko: 2
Rodzic: 8
Dziecko: 3
Rodzic: 9
Dziecko: 4
Rodzic: 10
Dziecko: 5
root@debian:~#
```

Zadanie 7

Program tworzy potok funkcją pipe() oraz proces potomny przez fork(). Rodzic zapisuje dane do fd[1], a dziecko odczytuje je z fd[0] (read()) i wypisuje w konsoli. Każdy proces zamyka nieużywany koniec potoku.

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main()
{
int fd[2];
pipe(fd);
pid_t pid = fork();

if (pid == 0)
{
close(fd[1]);
char buf[100];
read(fd[0], buf, sizeof(buf));
printf("Dziecko odczytalo: %s\n", buf);
close(fd[0]);
} else {
close(fd[0]);
char msg[] = "Wiadomosc od rodzica";
write(fd[1], msg, strlen(msg) + 1);
close(fd[1]);
}
return 0;
}
```

```
root@debian:~# gcc zad7L2.c -o zad7L2
root@debian:~# ./zad7L2
root@debian:~# Dziecko odczytalo: Wiadomosc od rodzica
```

Zadanie 8

Program demonstruje funkcję sleep() w obu procesach z różnymi czasami uśpienia. Dzięki temu widać, że procesy wykonują się asynchronicznie kończą pracę niezależnie od siebie, a kolejność komunikatów zależy od czasu uśpienia.

```
  GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
pid_t pid = fork();

if (pid == 0)
{
printf("Dziecko: startuje\n");
sleep(3);
printf("Dziecko: skonczylo po 3s\n");
} else {
printf("Rodzic startuje\n");
sleep(1);
printf("Rodzic: skonczylo po 1s\n");
wait(NULL);
}
return 0;
}
```

```
root@debian:~# gcc zad8L2.c -o zad8L2
root@debian:~# ./zad8L2
Rodzic startuje
Dziecko: startuje
Rodzic: skonczylo po 1s
Dziecko: skonczylo po 3s
root@debian:~# _
```

Zadanie 9

Proces dziecka wysyła sygnał SIGUSR1 do rodzica funkcją kill(getppid(), SIGUSR1). Rodzic wcześniej rejestruje prosty handler (signal() lub sigaction()), który po odebraniu sygnału wypisuje komunikat.

```
 GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handler(int sig)
{
printf("Rodzic: otrzymal sygnal SIGUSR1\n");
}

int main()
{
signal(SIGUSR1, handler);
pid_t pid = fork();

if (pid == 0)
{
sleep(1);
printf("Dziecko: wysylam SIGUSR1\n");
kill(getppid(), SIGUSR1);
} else {
wait(NULL);
printf("Rodzic: koniec\n");
}
return 0;
}
```

```
root@debian:~# gcc zad9L2.c -o zad9L2
root@debian:~# ./zad9L2
Dziecko: wysylam SIGUSR1
Rodzic: otrzymal sygnal SIGUSR1
Rodzic: koniec
root@debian:~#
```